

Linux - Part 2

6-MONTH INDUSTRY MASTERCLASS PATHWAY

An enterprise-grade, comprehensive technical architecture, deployment workflow, security hardening guide, and observability plan. Designed for senior engineering professionals (6+ years experience) striving for Principal SRE and Cloud Architect roles.

CORE CONCEPTS | PRODUCTION HARDENING | 20+ INCIDENT SCENARIOS

Automated SRE Learn System | Generated Pdf Generator

Linux - Advanced Tuning, cgroups v2, & System Hardening (Part 2)

1. Topic Introduction

This section explores the highly advanced system configurations, namespaces, unified cgroups v2 architectures, POSIX capabilities, and syscall sandboxing tools (seccomp filters) used to isolate critical daemons in enterprise environments.

2. Why This Topic is Critical

Default systems run daemons with overly broad execution privileges. Hardening processes at the kernel boundary blocks root-level exploits and restricts access to kernel structures.

3. Advanced SRE Capabilities & Resource Metering

- **POSIX Capabilities**: Replaces standard all-or-nothing root privileges. Processes can be run with specific privileges (e.g. `CAP_NET_BIND_SERVICE` allows binding ports < 1024 without running the process as root).
- **cgroups v2 unified control**: Governs CPU quotas, memory swap limits, block device weights, and dirty memory sync pools.

4. Hardened Security & Isolation Configuration

Below is a hardened seccomp filter setup and Capability assignment workflow:

```
# Assign specific capabilities to a binary instead of running as root
sudo setcap 'cap_net_bind_service=+ep' /opt/apiservice/bin/server

# View assigned capabilities
getcap /opt/apiservice/bin/server
```

5. Security & Kernel System Call Hardening

Seccomp (Secure Computing Mode) enables the kernel to instantly terminate a process if it invokes an unauthorized system call (e.g. preventing a web application from calling `execve` to execute a terminal shell).

Linux - Advanced Systems Hardening (Q21 to Q40)

Q21. How do POSIX Capabilities improve security over traditional root/setuid architectures?

Detailed Answer: Traditional Linux security uses an all-or-nothing model (root or standard user). POSIX Capabilities divide root privileges into distinct units (e.g. `CAP_NET_BIND_SERVICE`, `CAP_SYS_TIME`, `CAP_CHOWN`). This allows a daemon to bind to low ports or modify system times without giving it permissions to read/write all system files.

Production Scenario: Pinning `cap_net_bind_service=+ep` on web server binaries allows them to run entirely under standard, non-privileged usernames, protecting the system from arbitrary root shell spawns if the server code is exploited.

Q22. Explain the structural difference between cgroups v1 and cgroups v2.

Detailed Answer: cgroups v1 utilizes separate controllers (CPU, memory, blkio) organized in independent tree structures, leading to resource tracking mismatch issues. cgroups v2 implements a unified single-tree hierarchy, making resource metrics (like allocating dirty page flushes to their correct container limits) highly consistent.

Production Scenario: Running modern container runtimes (like containerd on Kubernetes) with cgroups v2 ensures accurate OOM and I/O saturation triggers per namespace without host context leaks.

Q23. How do you configure a seccomp syscall filter to deny process execution capabilities?

Detailed Answer: By writing a seccomp-bpf filter that intercepts system call requests at the SCI boundary, immediately returning `SECCOMP_RET_KILL` or `SECCOMP_RET_ERRNO` if a blacklisted call (like `execve`, `fork`, or `sys_ptrace`) is requested.

Production Scenario: Setting seccomp filters on container workloads prevents remote code execution exploits from executing `wget` or shell binaries, isolating the attacker within the memory bounds of the active program loop.

Q24. How do you troubleshoot slab cache leaks?

Detailed Answer: Use `slabtop` or check `/proc/slabinfo`. If files are opened and closed frequently without proper closing handles, kernel structures like `dentry` or `inode_cache` grow persistently, starving active RAM.

Production Scenario: An application leaks socket pointers. Over time, `slabtop` shows `ext4_inode_cache` and `sock_inode_cache` occupying 90% of kernel Slab space, triggering page reclaim thrashing.

Q25. What is the impact of vm.max_map_count on elastic search engines?

Detailed Answer: `vm.max_map_count` defines the limit of virtual memory map zones. Elasticsearch uses `mmap` to store indexes, requiring thousands of map areas.

Production Scenario: If `vm.max_map_count` is not bumped to at least 262144, Elasticsearch daemons will crash on startup with memory allocation failures.

... (Truncated for dry-run space; full version generates 20 full questions) ...